UNIVERSIDADE FEDERAL DE RONDÔNIA

Programação para Dispositivos Móveis

# Atividade Prática

## RecyclerView, Navegação e Imagens por URL

---

Prof. Dr. Lucas Marques da Cunha

2026

**Objetivo da atividade**

Modificar e ampliar o projeto de `RecyclerView` desenvolvido em aula, criando um aplicativo Android com três telas, listagem temática, carregamento de imagens por URL, navegação por `Intent`, tela de detalhes e remoção de itens por clique longo.

# Descrição da atividade

Utilize o código disponibilizado pelo professor como ponto de partida. O projeto original trabalha com uma lista de usuários, `RecyclerView`, `Adapter`, `ViewHolder`, clique simples e clique longo. Nesta atividade, você deverá adaptar essa estrutura para um novo tema.

Escolha um dos temas sugeridos: **Pokémon, países, filmes, animais, personagens, jogos, livros, pontos turísticos ou outro tema aprovado pelo professor**. O aplicativo deverá possuir **pelo menos 10 itens**.

Os dados textuais devem ficar armazenados no próprio código Java. **Não é necessário utilizar API, banco de dados ou serviço web**. A Internet será utilizada apenas para carregar as imagens por meio de URLs.

# Estrutura obrigatória

O aplicativo deverá possuir três `Activities`:

1. **Tela inicial**: nome do aplicativo, breve descrição e botão para acessar a listagem.
2. **Tela de listagem**: `RecyclerView` apresentando imagem e informações dos itens.
3. **Tela de informações**: exibe os dados completos do item selecionado.

Fluxo esperado:

$$\textbf{Tela Inicial} \longrightarrow \textbf{Listagem} \xrightarrow{\text{clique}} \textbf{Informações}$$

Na tela de listagem, o **clique longo** deverá remover o item e apresentar uma mensagem `Toast`.

# Tela inicial

A tela inicial deverá conter, no mínimo, o nome do aplicativo, uma breve descrição e um botão para acessar a listagem. A navegação deverá utilizar uma `Intent` explícita.

```
Intent intent = new Intent(MainActivity.this, ListaActivity.class);
startActivity(intent);
```

# Tela de listagem

A segunda tela deverá utilizar um `RecyclerView`. Cada item deverá exibir, no mínimo, uma imagem, o nome/título e uma informação complementar.

O projeto deverá utilizar `Adapter`, `ViewHolder`, `ArrayList` e um gerenciador de layout, como `LinearLayoutManager` ou `GridLayoutManager`.

### Clique simples

Ao clicar em um item, abra a tela de informações e envie os dados do objeto selecionado por `Intent`.

```
@Override
public void onItemClick(int position) {
    Pokemon p = lista.get(position);

    Intent intent = new Intent(ListaActivity.this,
                               DetalhesActivity.class);
    intent.putExtra("nome", p.getNome());
    intent.putExtra("tipo", p.getTipo());
    intent.putExtra("descricao", p.getDescricao());
    intent.putExtra("imagem", p.getImagem());
    startActivity(intent);
}
```

### Clique longo

O clique longo deverá remover o item e informar ao usuário qual elemento foi removido.

```
@Override
public void onItemLongClick(int position) {
    String nome = lista.get(position).getNome();
    lista.remove(position);
    adapter.notifyItemRemoved(position);

    Toast.makeText(ListaActivity.this,
            nome + " foi removido da lista.",
            Toast.LENGTH_SHORT).show();
}
```

## Tela de informações

A terceira tela deverá recuperar os dados recebidos pela `Intent` e apresentar a imagem e as informações completas do item.

```
String nome = getIntent().getStringExtra("nome");
String tipo = getIntent().getStringExtra("tipo");
String descricao = getIntent().getStringExtra("descricao");
String imagem = getIntent().getStringExtra("imagem");

// Exemplo:
txtNome.setText(nome);
txtTipo.setText(tipo);
txtDescricao.setText(descricao);
```

# Carregamento de imagens com Glide

Para carregar imagens por URL, utilize a biblioteca Glide.

No arquivo `build.gradle.kts` do módulo `app`:

```
dependencies {
    implementation("com.github.bumptech.glide:glide:4.16.0")
}
```

Ou, em projetos com `build.gradle` tradicional:

```
dependencies {
    implementation 'com.github.bumptech.glide:glide:4.16.0'
}
```

No `AndroidManifest.xml`, fora de `<application>`:

```
<uses-permission android:name="android.permission.INTERNET" />
```

Importe a biblioteca:

```
import com.bumptech.glide.Glide;
```

No `onBindViewHolder()` do `Adapter`:

```
Glide.with(holder.itemView.getContext())
        .load(lista.get(position).getImagem())
        .into(holder.imgItem);
```

Na tela de informações:

```
Glide.with(this)
        .load(imagem)
        .into(imgItem);
```

# Exemplos de dados

## Opção A - Pokémon

```
public class Pokemon {
    private String nome;
    private String tipo;
    private String descricao;
    private String imagem;

    public Pokemon(String nome, String tipo,
                   String descricao, String imagem) {
        this.nome = nome;
        this.tipo = tipo;
        this.descricao = descricao;
        this.imagem = imagem;
    }
```

```
    public String getNome() { return nome; }
    public String getTipo() { return tipo; }
    public String getDescricao() { return descricao; }
    public String getImagem() { return imagem; }
}
```

```
ArrayList<Pokemon> lista = new ArrayList<>();

lista.add(new Pokemon("Bulbasaur", "Planta / Veneno",
    "Possui uma semente plantada em suas costas.",
    "https://raw.githubusercontent.com/PokeAPI/sprites/master/
       sprites/pokemon/1.png"));

lista.add(new Pokemon("Charmander", "Fogo",
    "Possui uma chama na ponta de sua cauda.",
    "https://raw.githubusercontent.com/PokeAPI/sprites/master/
       sprites/pokemon/4.png"));

lista.add(new Pokemon("Squirtle", "Agua",
    "Utiliza seu casco para protecao.",
    "https://raw.githubusercontent.com/PokeAPI/sprites/master/
       sprites/pokemon/7.png"));

lista.add(new Pokemon("Pikachu", "Eletrico",
    "Armazena eletricidade em suas bochechas.",
    "https://raw.githubusercontent.com/PokeAPI/sprites/master/
       sprites/pokemon/25.png"));
```

## Opção B - Países

```
public class Pais {
    private String nome;
    private String capital;
    private String continente;
    private String imagem;

    public Pais(String nome, String capital,
                String continente, String imagem) {
        this.nome = nome;
        this.capital = capital;
        this.continente = continente;
        this.imagem = imagem;
    }

    public String getNome() { return nome; }
    public String getCapital() { return capital; }
    public String getContinente() { return continente; }
    public String getImagem() { return imagem; }
}
```

```
ArrayList<Pais> lista = new ArrayList<>();
```

```
lista.add(new Pais("Brasil", "Brasilia", "America do Sul",
        "https://flagcdn.com/w320/br.png"));
lista.add(new Pais("Argentina", "Buenos Aires", "America do Sul",
        "https://flagcdn.com/w320/ar.png"));
lista.add(new Pais("Canada", "Ottawa", "America do Norte",
        "https://flagcdn.com/w320/ca.png"));
lista.add(new Pais("Japao", "Toquio", "Asia",
        "https://flagcdn.com/w320/jp.png"));
lista.add(new Pais("Portugal", "Lisboa", "Europa",
        "https://flagcdn.com/w320/pt.png"));
```

## Opção C - Filmes

```
public class Filme {
    private String titulo;
    private String genero;
    private String ano;
    private String descricao;
    private String imagem;

    public Filme(String titulo, String genero, String ano,
                 String descricao, String imagem) {
        this.titulo = titulo;
        this.genero = genero;
        this.ano = ano;
        this.descricao = descricao;
        this.imagem = imagem;
    }

    public String getTitulo() { return titulo; }
    public String getGenero() { return genero; }
    public String getAno() { return ano; }
    public String getDescricao() { return descricao; }
    public String getImagem() { return imagem; }
}
```

```
lista.add(new Filme(
    "Nome do filme",
    "Ficcao cientifica",
    "2020",
    "Descricao do filme.",
    "URL_DA_IMAGEM"
));
```

Neste tema, pesquise URLs públicas adequadas para os pôsteres ou utilize imagens hospedadas em local acessível por URL.

## Opção D - Animais

```
public class Animal {
    private String nome;
```

```
    private String especie;
    private String habitat;
    private String descricao;
    private String imagem;

    public Animal(String nome, String especie, String habitat,
                  String descricao, String imagem) {
        this.nome = nome;
        this.especie = especie;
        this.habitat = habitat;
        this.descricao = descricao;
        this.imagem = imagem;
    }

    public String getNome() { return nome; }
    public String getEspecie() { return especie; }
    public String getHabitat() { return habitat; }
    public String getDescricao() { return descricao; }
    public String getImagem() { return imagem; }
}
```

## Adaptação do projeto fornecido

O objetivo não é criar um projeto independente do código trabalhado em aula. Identifique e modifique as partes necessárias. Por exemplo, substitua:

```
ArrayList<Usuario>
```

por uma lista correspondente ao tema escolhido:

```
ArrayList<Pokemon>
// ou ArrayList<Pais>, ArrayList<Filme>, etc.
```

No código original, a imagem é um recurso local:

```
holder.imgAvatar.setImageResource(list.get(position).getFigura());
```

Como a nova imagem será uma URL, substitua essa lógica pelo Glide:

```
Glide.with(holder.itemView.getContext())
        .load(list.get(position).getImagem())
        .into(holder.imgAvatar);
```

## Requisitos mínimos

1. possuir três `Activities`;
2. possuir tela inicial com acesso à listagem;
3. utilizar `Intent` para navegação;
4. utilizar `RecyclerView` na segunda tela;

5. possuir pelo menos 10 itens;
6. utilizar uma classe de modelo adequada ao tema;
7. apresentar imagem e informações textuais em cada item;
8. carregar imagens remotas utilizando Glide;
9. implementar clique simples nos itens;
10. abrir a tela de informações após o clique;
11. enviar os dados do item selecionado por `Intent`;
12. apresentar na terceira tela a imagem e as informações recebidas;
13. implementar clique longo;
14. remover o item selecionado após o clique longo;
15. atualizar corretamente o `RecyclerView`;
16. apresentar um `Toast` informando qual item foi removido.

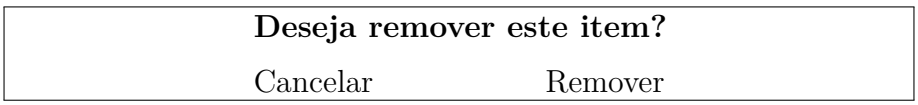
## Organização sugerida

```
java/
  MainActivity.java
  ListaActivity.java
  DetalhesActivity.java
  Pokemon.java          // ou classe equivalente
  PokemonData.java      // opcional
  Adapter.java

res/layout/
  activity_main.xml
  activity_lista.xml
  activity_detalhes.xml
  layout_item.xml
```

## Desafio adicional

Como extensão opcional, implemente um `AlertDialog` antes da exclusão. O item deverá ser removido somente após a confirmação do usuário.




## Entrega

Entregue o projeto Android completo, organizado e executável. O código deverá estar coerente com o tema escolhido e atender aos requisitos mínimos apresentados neste documento.